

Draw It or Lose It Game Application

CS 230 Project Software Design Template

Version 1.3

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	9/18/2026	Titus Lamb	Written Executive Summary, Requirements, Design Constraints, and System Architecture View.
1.1	9/19/2026	Titus Lamb	Written Domain Model.
1.2	10/04/2026	Titus Lamb	Completed Evaluation Table.
1.3	10/15/2026	Titus Lamb	Completed Recommendations.

Executive Summary

The Gaming Room, an entertainment company, wants to develop a web-based game that can run on multiple platforms that are based off their original game, *Draw It or Lose It*. *Draw It or Lose It* is currently only available in the Android App store.

Requirements

- The game must have the ability to host one or more teams.
- Each team will have multiple players.
- Game and team names will be unique, so players can check if a name is already in use when creating a new team name.
- Only one game can exist in memory at a time.

Design Constraints

1. Compatibility Constraint
Draw It or Lose It currently only exists in the Android App store. Transferring this application to the web requires it to run on browsers and other operating systems that the Android version did not have to do. This also means users using different operating systems need to be able to play the same games. Any old application code that is Android specific cannot be used in the new version. Different operating systems also handle input detection differently. This doesn't only apply to operating systems but also to different web browsers. When it comes to the implications on the development, making sure you are using the correct language (for this assignment we default to java) is essential for cross compatibility. An API must be created to handle different inputs that all lead to the same response depending on the device used. UI needs to be responsive to different screen sizes and dimensions because of the large array of devices that could be used.

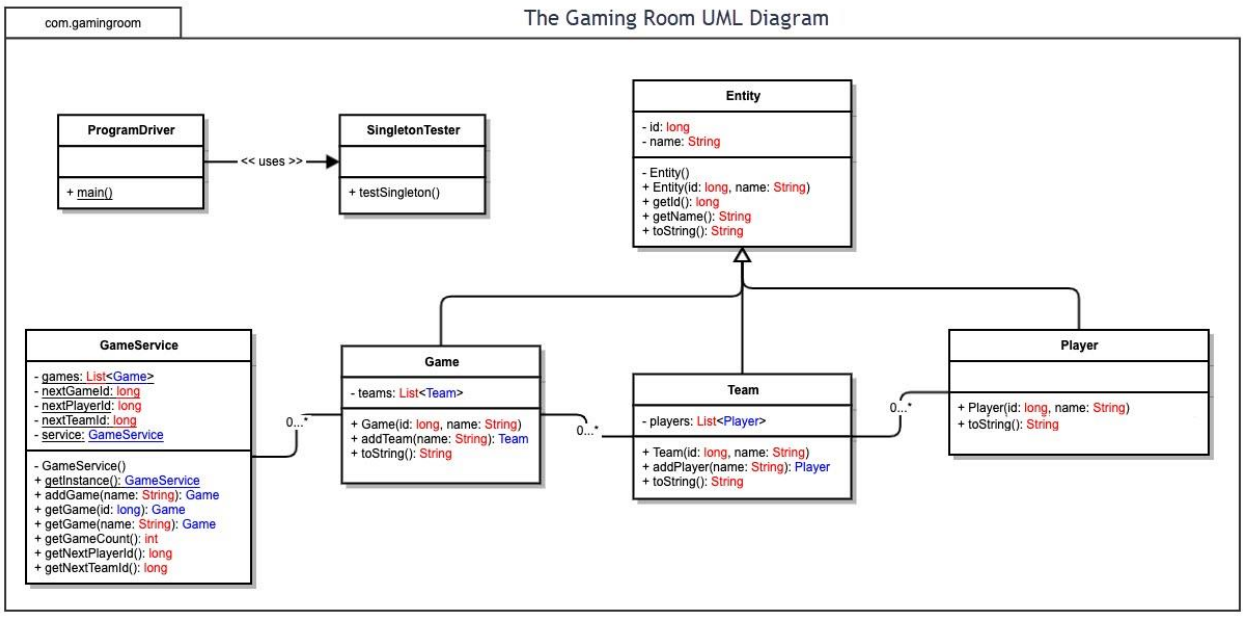
2. Performance Constraint
Not only being a web application but also needing to stream the rendered images simultaneously puts a hard constraint on the performance of the application. Because the game must be in memory as a single instance, it is a shared resource between all players. Specific devices such as older iPhones have very limited computing power, which can cause stuttering or breaking if too many resources are used in the application. For the implications it has on development, the game must have a singleton class approach to enforce the one game object in memory at a time rule. Images must be streamed efficiently to not overload a user's resources. We need to stress test the application to find any bottlenecks with player count and image streaming.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

This is *The Gaming Room UML Diagram* for the singleton game application *Draw It or Lose It*. We have a driver class called ProgramDriver that only has one method main() that acts as the entry point to the application. We have a test class called SingletonTester that holds the method testSingleton(), which is made to check that only one instance of the game exists. For the main classes we have Entity, Game, Team, and Player. Entity is the base class for the other classes. It can be called to by using the constructor Entity() and can be interacted with using its multiple getters such as getId(), getName(), and toString(). Game inherits Entity and associates with Team as one Game can contain zero or many Teams. Team inherits Entity and associates with Player as one team can contain zero or many Players. Player also inherits Entity. Many object-oriented principles are demonstrated such as inheritance between Entity and the three other classes of Game, Team, and Player. Another would be encapsulation as all attributes are private and can only be accessed from controlled public methods. This also makes sure entity IDs are all unique. Polymorphism can also be seen as Entity, Game, Player, and Team all override toString(). This means that the classes inheriting Entity can still have their own unique outputs. Lasty, the singleton class approach can be seen with GameService having its own private constructor GameService(), its private attribute service, and getInstance() as the only way to access the instance making sure only one instance can exist. SingletonTester helps by making sure only one instance returns.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Runs web servers. MacOS is costly when taking into account apple hardware. Cloud accessibility is limited, and scaling is poor. No immediate licensing costs, but Mac hardware is required.	Known to run servers well and is the most widely used host. Very low costs due to flexibility in hardware. Large cloud accessibility and great scaling. No licensing costs.	Runs web servers well but can be costly due to licensing. Many choices for Cloud, but licensing rates are costly. Hardware can be flexible, and scaling is good.	Not built to run web servers. Hardware and cloud use are very limited. Terrible scaling due to the OS suspending background processes (simply not made to run servers), limited resources, and poor performance.
Client Side	High costs due to testing (Need apple hardware). Will need higher expertise to work with the web due to high security requirements and unique developing ecosystems.	Extremely low costs. Testing is easy to start, and support is large for learning. Smaller testing time and attention needed due to small user counts.	Low costs. Most widely used when it comes to desktops. Testing needs to be thorough and could be time costly. Low expertise is needed.	High fees due to multiple tools and software needed for testing. Large flexibility and responsiveness are required for touch screens and screen sizes. Usually, it is worth due to the highest number of users. High expertise is needed and very time costly.
Development Tools	Languages used are HTML, CSS, JavaScript, Swift, and even C/C++. The biggest IDEs used are VS Code and Xcode.	Languages used are C/C++, Rust, Bash, Python, JavaScript, and many more. Overwhelming use of VS Code for an IDE. Tools like Git and Docker are commonly used. Mostly open source.	Languages used are C#/.NET, C++, JavaScript, and Python. VS Code takes top spot in IDE used with Microsoft Visual Studio (Paid tiers) and PyCharm also being used.	Languages used are mostly Kotlin and Swift, with also the use of JavaScript. Web work specifically requires HTML, CSS, and JavaScript. IDEs used are Android Studio and Xcode with both native. Multiple teams would be needed for both native tools.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: For an operating platform, I recommend Linux. With no operating system licensing costs when compared to other alternatives (Linux.com, 2018), and flexibility with many cloud providers, it can be a great choice. Linux does not have specific hardware requirements allowing flexibility if hardware needs to be changed due to scaling.

2. **Operating Systems Architectures**: With the kernel as the systems core, Linux has full control over what can run and when. Linux has direct control over memory and storage, and what can be allocated to each process. This is due to its very modular architecture where pieces of the OS can be disabled and still function. The game will need direct permission from the kernel to run any processes and can prevent spreading any issues if the game were to crash.

3. **Storage Management**: For storing data such as player information, teams, scores, and other future features, it will be put in a database such as MySQL. Databases such as the one provided are efficient and organized. This keeps data accurate, handles multiple requests at once, and pulls data quickly.

4. **Memory Management**: Because Linux has specific control over its memory, each part of the game will have its own share of available memory given to them. When a game ends, its memory is cleared to be used for the next game. For data that is rarely used but still important for the game to function, it can be sent to storage.

5. **Distributed Systems and Networks**: Clients will send requests to the server (Linux) and back using JSON. Due to the simplicity of the data that's sent back and forth as well as being the industry standard, the platform the client is using does not matter. To handle large amounts of requests and to tackle server outages, we can have multiple servers handle requests evenly so if one fails, the others are still up. Each server can connect to a single database for synced data, which is a heavy dependency. Having backups put in place is a must. Due to the system requiring clients to have a good connection, it is essential we handle failed client requests by retrying to send data to the client. Players should also be given the option to rejoin games upon disconnecting.

6. **Security**: For security, all traffic will be encrypted using HTTPS to keep client data protected. Player account information can be salted and then hashed before being put into the database to ensure protected passwords if the database were to be breached. All requests will require authentication through REST API, and authorization will be used to make sure users can only have access to requesting their own data. All inputs will be validated on the server to prevent bad actors from sending false requests. Linux always has full control over who can access what program and has the ability to set file permissions. With this as well as consistent security software updates and a firewall that comes with Linux, it is powerful for keeping permissions minimal and shutting down bad actors from accessing important features or data.

References

Linux.com. (2018, May 31). *What is a linux server and why does your business need one?* Linux.com. https://www.linux.com/news/what-linux-server-and-why-does-your-business-need-one-0/